Project Report – CS 331

AES Encryption/Decryption Showcase

Audrey Fitchett, Braedon Kennedy, Cesar Ochoa, Leo Haener-Pope, Rhett Edwards, Runda Li

CS 331 Computer Security

Professor Sindu Reddy Kalathur Gopal

December 8, 2025

GitHub Repository Link: https://github.com/leohaenerpope/CS-331-Group-5-Project

Abstract

This project is a demonstration of the Advanced Encryption System (AES) method of cryptography and how it handles the encryption and decryption of texts. Our main objectives were gaining a concrete understanding of how AES works as a cryptographic system and why it is a valuable part of the cybersecurity world. We wanted to highlight how AES worked through creating a simple Java-based application that performed all necessary steps for AES encryption and decryption.

We learned there are many different types of AES encryption, including both the use of different size keys, and also different encryption modes. For our project, we utilized a simple 128-bit key size, as opposed to a 192-bit or 256 bit key size. We also utilized the Electronic Codebook (ECB) encryption mode, which breaks texts into fixed size blocks for the encryption and decryption processes. This is a very simple method, and is not used in wide-spread practice as it isn't as secure as other encryption modes. For this project, we deemed it beneficial as it allows for us to place more emphasis on covering how the main processes of AES work, which was our main goal.

Ultimately, we gained a substantial understanding of how AES cryptography works, though functions that utilize confusion and diffusion to create a strong cryptosystem. We can apply and adapt our main encryption/decryption functions to real-life cybersecurity scenarios where AES cryptography is used to keep information private.

Introduction

We saw the need for this project coming from the increasing need to secure digital information as more communication and data storage move online. Unencrypted data can be easily intercepted and exploited. Secure cryptographic methods are essential for maintaining information integrity. Our project addresses this issue by showing how the Advanced Encryption Standard (AES) converts readable information into protected ciphertext. Our aim is to demonstrate why encryption is a critical defense in modern cybersecurity.

A short summary of AES is that it is simply a form of symmetric cryptography, using a shared single key for both encryption and decryption. Utilizing processes that confuse and diffuse a private message, it is an essential tool in a secure cybersecurity environment. The system takes in a given input and pre-determined base key. Using the base key, it initializes round keys for each round of the process. For a 128-bit key, a total of 10 rounds are used. In each round, the functions ByteSubstitution, ShiftRows, MixColumns, and AddRoundKey are utilized. The ByteSubstitution function sends bytes of a 128-bit input through a predetermined hex value table that confuses the data, making it harder to spot patterns in the text. Then, ShiftRows, which shifts the rows of a 4x4 table of the 16 bytes (128 bits, 16 bytes), and MixColumns, which mixes bytes within columns using multiplication. These two functions diffuse the data, ensuring a single change in the input affects many parts of the output. Finally, the RoundKey function is used to XOR the new bits with the roundKey

corresponding to the current round, providing more confusion to the data. Also, before the first round, a round zero key is added, and the last round does not utilize MixColumns. Decryption is fairly simple as it is simply the reverse operation of encryption [2].

AES is used in a wide range of real world applications. They can vary from secure messaging, financial systems, and cloud storage. Through implementing a simplified AES encryption and decryption program, we gained practical insight into how these systems protect sensitive information. The report will cover the background of AES, the methodology used to design and implement our program, the results of our testing, and the overall significance of our findings within the broader context of computer security.

Introduction – Member Contributions

Leo Haener-Pope worked on creating the KeyManager file which contained the process for generating round keys, providing information for the main AES encryption/decryption AddRoundKey function. Also added some parts to the AES encryption/decryption functions, including both the Mix Columns and Inverse Mix Columns byte multiplication step.

Cesar Ochoa worked on implementing the AES decryption process for the project. He worked on the decryption method and added helper functions such as InvShiftRows, InvByteSubstitution, and InvMixColumns to reverse the encryption steps. This was used to ensure that encrypted messages could be accurately converted back into plaintext, completing the full encryption–decryption workflow of the program.

Braedon Kennedy worked on implementing the AES encryption process. He wrote the basic AES file and constructor and implemented the encryption method and each of its helper functions. He designed it so that the encryption and decryption methods should manage larger strings by the acceptable size at a time, this was all he did in the decryption method.

Runda Li implemented the `MainProject` driver program and integrated the AES and `KeyManager` classes into a working command-line demo. He added user input handling, key-length validation, and test cases, and debugged issues in the encryption/decryption flow so that the program could reliably show an end-to-end “encrypt → decrypt” demonstration.

Audrey Fitchett implemented the output for the program. She added an option for arguments to the MainProject driver program and added output for each step of the process in both the encryption and decryption process for AES. She also fixed bugs as they were revealed by the step by step demonstration output and tested the program with keys and messages of various lengths.

Rhett Edwards implemented the AESTester module. This module utilized the Java library plugins available for AES encryption. This ensured that the work being done on the other classes could be verified against a known working example. This was all tested converting to Base64.

Literature Review / Background

The project is based on the mechanics behind the AES, which uses mathematical operations. Substitution, permutation, and linear algebra applied over finite fields builds up the core of the encryption method. In real-world systems it is implemented using more complex modes, key sizes, and security measures designed to address side channel and timing attacks. In contrast our project simplifies these elements using basic linear algebra and the ECB mode to focus on demonstrating how the essential encryption and decryption processes operate. This approach provides a clearer educational view of the AES mechanics to be explored step by step.

Objectives

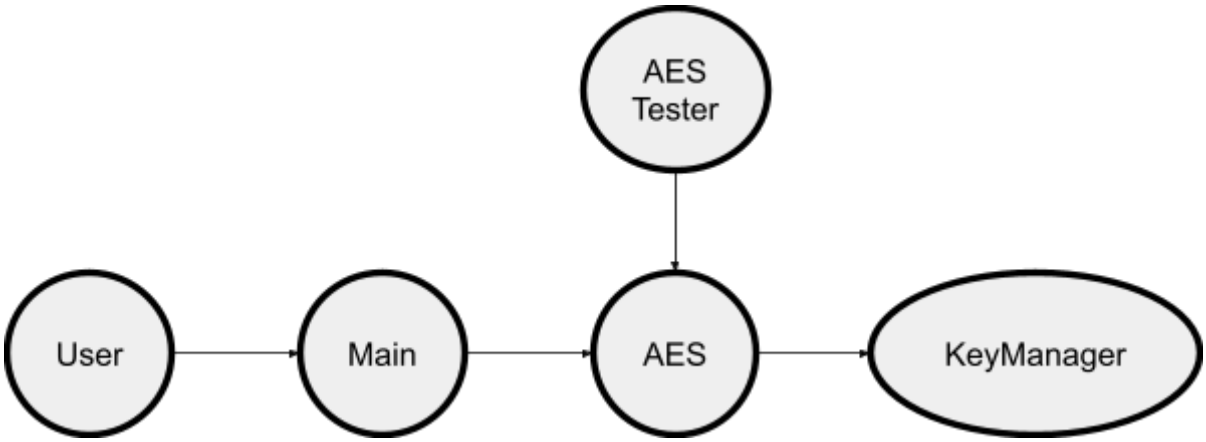
- Demonstrate the encryption and decryption of a message using AES cryptography using a Java program

This project is aimed to demonstrate the encryption and decryption of a message using AES cryptography, which allows both us and others to gain a further understanding of how the cryptography works. Understanding how each function and step of AES works is key to highlighting how it uses diffusion and confusion to ensure the security of private messages.

- Highlighting why AES cryptography is a key component to computer security

Another objective of ours was to gain a significant knowledge as to why both AES and cryptography in general are essential parts of computer security. Through the process of first designing our program, then implementing and testing it, we have experienced both successes and failures which have all led to a firmer understanding of how and why cryptographic methods like AES are used in practice for cyber security environments.

System Design / Methodology



Project Design Modules

- Main
    - Utilizes a scanner to receive input from the user and then provides corresponding AES output
- AES
    - Variables
        - roundNum
            - The number of rounds in the encryption/decryption process
        - currentRoundNum
            - Which round the program is currently in
        - keyManager
            - Object that stores all key and round key information
        - dataArray
            - The data in the form of a 4x4 array of doubles
        - dataMatrix
            - The data in the form of a 4x4 matrix of doubles
        - mixedColArr
            - A preset array to mix the columns with
        - byteSubstitutionMatrix
            - A present array to substitute bytes with
    - Constructor
        - No Parameters
        - Creates an AES Object that has 10 rounds and a present byte substitution matrix
    - Encrypt()/Decrypt()
        - Data: message to encrypt/decrypt, Key: the AES key to use
        - Runs a provided string and encrypts/decrypts it correspondingly to the provided key string
    - ShiftRows() and Inverse
        - Private helper function
        - No Parameters
        - Shifts each row in the data matrix by the row number (1-4)

    - MixColumns() and Inverse
        - Private helper function
        - No Parameters
        - Multiplies the data Matrix by a corresponding mix or unmix matrix
    - ByteSubstitution() and Inverse
        - Private helper function
        - No Parameters
        - Substitutes each byte in the matrix by a corresponding substitution matrix
    - AddRoundKey()
        - Private helper function
        - No Parameters
        - Performs an exclusive or operation on each byte in the matrix with the round key provided by the keyManager
- KeyManager
    - Variables
        - key
            - A byte array of the key
        - keySize
            - The size of the key in bits
        - stringKeySize
            - The size of the key in bytes
        - roundKeys
            - Array of each round key
        - SBOX
            - 1 dimensional substitution array
    - Constructor
        - Generates key if none is provided
        - Limits string input to proper size and converts it to bytes, it then generates the round keys
    - generateRoundKeys()
        - Private helper function
        - key: initial key to use
        - Returns a 2 dimensional array of bytes for each rounds key
    - keyScheduleStep()
        - Private helper function
        - startKey: provided previous key input, round: which round the it is on
        - Generates a round key
    - keyScheduleXOR()
        - Private helper function
        - first: First byte array to be XORed, second: Second byte array to be XORed
        - Used when key scheduling, XORs two different byte arrays, both of length 4 Individually XORs each specific spot in the 2 byte arrays
    - keyScheduleGFunction()
        - Private helper function

        - inputBytes: The 4 bytes to be put through the G Function, round: The current round of key scheduling, necessary for round coefficient function
        - The G function for key scheduling, used to determine new first 4-byte long word
    - keyScheduleSBoxFunction()
        - Private helper function
        - inputByte: Byte to be put through s-box
        - Puts a byte through the AES S-box function for key scheduling and returns the applied byte.
    - keyScheduleRoundCoefficient()
        - Private helper function
        - round: Current round of key scheduling, necessary for correctly chosen round coefficient to XOR inputByte with
        - Returns round coefficient function for key scheduling, used in the G Function on one byte.
    - getRoundKeys()
        - Returns a 2 dimensional array of the round keys
    - getRoundKey()
        - Returns an array corresponding to a round key
    - printKey()
        - Print the key converted into bytes to the console
    - printRoundKeys()
        - Prints out each round key to the console

Programming Languages

- Java

Tools

- VSCode
    - Inline comment suggestions by copilot
- Copilot, ChatGPT
    - Generation of values for AES S-BOX Jama Matrixes and Java Byte Arrays to avoid the manual input for each value of the 16x16 table sets of pre-determined byte values (See Implementation code in AES Java constructor and KeyManager).

Libraries

- JAMA
    - Java Matrix Package
    - Provides basic linear algebra use

Implementation

Implementation was the same as the design but we added more methods and some debugging methods.

The additional methods are:

- AES
  - [1] gmul()

```
        /**
         * Galois Field Multiplication that is used to multiply the bytes in the
         * MixColumns and InverseMixColumns functions
         *
         * When AES multiplies bytes in the mix columns function,
         * it uses this type of multiplication instead of regular multiplication
         *
         * It is basically just like a form of binary multiplication
         * but using XOR as addition/subtraction and reducing by a modulus to make
         * sure result stays below 8 bits
         *
         * For more info
         * https://www.youtube.com/watch?v=oPjkfvuqqv4
         *
         * @param a int of 8 bits to be multiplied
         * @param b int of 8 bits to be multiplied
         * @return result of Galois Field Multiplication
         */
        private int gmul(int a, int b) {
            int p = 0;
            for (int i = 0; i < 8; i++) {
                if ((b & 1) != 0) {
                    p ^= a;
                }
                boolean highBit = (a & 0x80) != 0; // checks for if highest bit is 1 (result
    becomes larger than 8 bits, so subtract modulus)
                a = (a << 1) & 0xFF;
                if (highBit) {
                    a ^= 0x1b; // this subtracts the modulus, XOR is the same as subtraction
    in Galois Field
                }
                b >>= 1;
            }
            return p & 0xFF;
        }
```

  - PrintMatrix()

```
        public void PrintMatrix(){
            double[][] printArray = dataMatrix.getArray();

                    for(int row = 0; row < 4; row ++){
                        for(int col = 0; col < 4; col++){
                            if(col != 3){
                                System.out.print(printArray[row][col] + " ");
                            }
                            else{
                                System.out.println(printArray[row][col]);
                            }
                        }
                    }
        }
```

- KeyManager
  - setCorrectStringKeySize()

```
        /**
         * Makes sure that the given key string for KeyManager constructor is the
    correct size,
         * which for 128 bit key would be 128 / 8 = 16
         *
         * @param keyString String to check
```

```
         * @return The correct size of string, either padded with characters, cut, or
    the same
         */
        private String setCorrectStringKeySize(String keyString){
            if (keyString.length() > stringKeySize){
                keyString = keyString.substring(0, stringKeySize);
                System.err.println("Given key string size was longer than " +
    stringKeySize + ". Setting new key to: " + keyString);
            }else if (keyString.length() < stringKeySize){
                while (keyString.length() < stringKeySize){
                    keyString = keyString + "0";
                }
                System.err.println("Given key string size was shorter than " +
    stringKeySize + ". Setting new key to: " + keyString);
            }
            return keyString;
        }
```

- getByteKeyFromString()

```
        /**
         * Processes a string into a byte array.
         *
         * @param key key string to set KeyManager key variable to
         */
        private byte[] getByteKeyFromString(String key){
            byte[] newKey = key.getBytes(StandardCharsets.UTF_8);
            if (newKey.length != keySize/8){
                System.err.println("WARNING: Bad key size being used in
    KeyManager.java");
            }
            return newKey;
        }
```

Code Snippets

AES variables and constructor:

```
    private int roundNum;
    private int currentRoundNum;
    private KeyManager keyManager;
    private double[][] dataArray;
    private double[][] mixedColumnArr = {
        {2, 3, 1, 1},
        {1, 2, 3, 1},
        {1, 1, 2, 3},
        {3, 1, 1, 2}
    };
    private Matrix dataMatrix;
    private Matrix byteSubstitutionMatrix;

    public AES() {
        this.roundNum = 10;
        this.currentRoundNum = 0;

        this.dataArray = new double[4][4];

        this.byteSubstitutionMatrix = new Matrix(new double[][]{
            { 0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76 },
            { 0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2, 0xaf, 0x9c, 0xa4, 0x72, 0xc0 },
            { 0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5, 0xf1, 0x71, 0xd8, 0x31, 0x15 },
            { 0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80, 0xe2, 0xeb, 0x27, 0xb2, 0x75 },
            { 0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6, 0xb3, 0x29, 0xe3, 0x2f, 0x84 },
            { 0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe, 0x39, 0x4a, 0x4c, 0x58, 0xcf },
            { 0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02, 0x7f, 0x50, 0x3c, 0x9f, 0xa8 },
            { 0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda, 0x21, 0x10, 0xff, 0xf3, 0xd2 },
            { 0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e, 0x3d, 0x64, 0x5d, 0x19, 0x73 },
            { 0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8, 0x14, 0xde, 0x5e, 0x0b, 0xdb },
            { 0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac, 0x62, 0x91, 0x95, 0xe4, 0x79 },
            { 0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4, 0xea, 0x65, 0x7a, 0xae, 0x08 },
            { 0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74, 0x1f, 0x4b, 0xbd, 0x8b, 0x8a },
            { 0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57, 0xb9, 0x86, 0xc1, 0x1d, 0x9e },
            { 0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87, 0xe9, 0xce, 0x55, 0x28, 0xdf },
            { 0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d, 0x0f, 0xb0, 0x54, 0xbb, 0x16 }
        });
```

```
        this.dataMatrix = new Matrix(dataArray);
    }
```

Encryption Method:

```
public String encrypt(String data, String key) {
        this.keyManager = new KeyManager(key);
        keyManager.printKey();
        keyManager.printRoundKeys();

        String fullEncryptedData = "";
        for (int d = 0; d < data.length(); d += 16) {
            String dataBlock = data.substring(d, Math.min(data.length(), d+16));
            while (dataBlock.length() < 16) {
                dataBlock += "\0"; // padding with null characters
            }
            this.currentRoundNum = 0;
            this.dataArray = new double[4][4];
            for (int i = 0; i < dataBlock.length(); i++) {
                int row = i % 4;
                int col = i / 4;
                dataArray[row][col] = dataBlock.charAt(i);
            }
            this.dataMatrix = new Matrix(dataArray);
            AddRoundKey();
            for (currentRoundNum = 1; currentRoundNum < roundNum; currentRoundNum++) {
                ByteSubstitution();
                ShiftRows();
                MixColumns();
                AddRoundKey();
            }
            ByteSubstitution();
            ShiftRows();
            AddRoundKey();

            // convert matrix back into ciphertext
            dataArray = dataMatrix.getArray();
            byte[] cipherBytes = new byte[16];
            for (int i = 0; i < 16; i++) {
                int row = i % 4;
                int col = i / 4;
                cipherBytes[i] = (byte) dataArray[row][col];
            }

            // convert ciphertext into base64 string and store in var
            String encryptedData =
java.util.Base64.getEncoder().encodeToString(cipherBytes);

            fullEncryptedData += encryptedData;
        }
        return fullEncryptedData;
    }
```

Decryption Method:

```
    public String decrypt(String encryptedData, String key) {
        // use key from key manager
        this.keyManager = new KeyManager(key);
        this.currentRoundNum = roundNum; // start at round 10 (final)

        String fullDecryptedData = "";
        for (int d = 0; d < encryptedData.length(); d += 24) {
            String dataBlock = encryptedData.substring(d,
```

```
Math.min(encryptedData.length(), d+24));
            while (dataBlock.length() < 16) {
                dataBlock += "\0"; // padding with null characters
            }

            byte[] cipherBytes = java.util.Base64.getDecoder().decode(dataBlock);

            // Load cipher into 4x4 matrix
            for (int i = 0; i < 16; i++) {
                int row = i % 4;
                int col = i / 4;
                dataArray[row][col] = (i < cipherBytes.length) ? cipherBytes[i] & 0xFF :
0;
            }
            dataMatrix = new Matrix(dataArray);
            AddRoundKey(); // final round key

            // inverse transformation, for final [inverse shiftrows + inverse subbytes]
            InvShiftRows();
            InvByteSubstitution();

            // run the decryption for round 1-9
            for (currentRoundNum = roundNum - 1; currentRoundNum > 0; currentRoundNum--)
{
                AddRoundKey();
                InvMixColumns();
                InvShiftRows();
                InvByteSubstitution();
            }
            AddRoundKey(); //round 0 key

            // convert matrix back into plaintext
            dataArray = dataMatrix.getArray();
            byte[] plainBytes = new byte[16];
            for (int i = 0; i < 16; i++) {
                int row = i % 4;
                int col = i / 4;
                plainBytes[i] = (byte) dataArray[row][col];
            }

            // convert plaintext into str and store in var
            String decryptedData = new String(plainBytes).trim();
            fullDecryptedData += decryptedData;
        }
        return fullDecryptedData;
    }
```

KeyManager variables and constructor:

```
    private byte[] key; // byte array used to store the key. Should have a length of 16.
    private int keySize = 128;
    private int stringKeySize = keySize / 8;
    private byte[][] roundKeys;
    private static final byte[] SBOX = new byte[] {

(byte)0x63,(byte)0x7c,(byte)0x77,(byte)0x7b,(byte)0xf2,(byte)0x6b,(byte)0x6f,(byte)0xc5,

(byte)0x30,(byte)0x01,(byte)0x67,(byte)0x2b,(byte)0xfe,(byte)0xd7,(byte)0xab,(byte)0x76,

(byte)0xca,(byte)0x82,(byte)0xc9,(byte)0x7d,(byte)0xfa,(byte)0x59,(byte)0x47,(byte)0xf0,
```

```
(byte)0xad,(byte)0xd4,(byte)0xa2,(byte)0xaf,(byte)0x9c,(byte)0xa4,(byte)0x72,(byte)0xc0,
(byte)0xb7,(byte)0xfd,(byte)0x93,(byte)0x26,(byte)0x36,(byte)0x3f,(byte)0xf7,(byte)0xcc,
(byte)0x34,(byte)0xa5,(byte)0xe5,(byte)0xf1,(byte)0x71,(byte)0xd8,(byte)0x31,(byte)0x15,
(byte)0x04,(byte)0xc7,(byte)0x23,(byte)0xc3,(byte)0x18,(byte)0x96,(byte)0x05,(byte)0x9a,
(byte)0x07,(byte)0x12,(byte)0x80,(byte)0xe2,(byte)0xeb,(byte)0x27,(byte)0xb2,(byte)0x75,
(byte)0x09,(byte)0x83,(byte)0x2c,(byte)0x1a,(byte)0x1b,(byte)0x6e,(byte)0x5a,(byte)0xa0,
(byte)0x52,(byte)0x3b,(byte)0xd6,(byte)0xb3,(byte)0x29,(byte)0xe3,(byte)0x2f,(byte)0x84,
(byte)0x53,(byte)0xd1,(byte)0x00,(byte)0xed,(byte)0x20,(byte)0xfc,(byte)0xb1,(byte)0x5b,
(byte)0x6a,(byte)0xcb,(byte)0xbe,(byte)0x39,(byte)0x4a,(byte)0x4c,(byte)0x58,(byte)0xcf,
(byte)0xd0,(byte)0xef,(byte)0xaa,(byte)0xfb,(byte)0x43,(byte)0x4d,(byte)0x33,(byte)0x85,
(byte)0x45,(byte)0xf9,(byte)0x02,(byte)0x7f,(byte)0x50,(byte)0x3c,(byte)0x9f,(byte)0xa8,
(byte)0x51,(byte)0xa3,(byte)0x40,(byte)0x8f,(byte)0x92,(byte)0x9d,(byte)0x38,(byte)0xf5,
(byte)0xbc,(byte)0xb6,(byte)0xda,(byte)0x21,(byte)0x10,(byte)0xff,(byte)0xf3,(byte)0xd2,
(byte)0xcd,(byte)0x0c,(byte)0x13,(byte)0xec,(byte)0x5f,(byte)0x97,(byte)0x44,(byte)0x17,
(byte)0xc4,(byte)0xa7,(byte)0x7e,(byte)0x3d,(byte)0x64,(byte)0x5d,(byte)0x19,(byte)0x73,
(byte)0x60,(byte)0x81,(byte)0x4f,(byte)0xdc,(byte)0x22,(byte)0x2a,(byte)0x90,(byte)0x88,
(byte)0x46,(byte)0xee,(byte)0xb8,(byte)0x14,(byte)0xde,(byte)0x5e,(byte)0x0b,(byte)0xdb,
(byte)0xe0,(byte)0x32,(byte)0x3a,(byte)0x0a,(byte)0x49,(byte)0x06,(byte)0x24,(byte)0x5c,
(byte)0xc2,(byte)0xd3,(byte)0xac,(byte)0x62,(byte)0x91,(byte)0x95,(byte)0xe4,(byte)0x79,
(byte)0xe7,(byte)0xc8,(byte)0x37,(byte)0x6d,(byte)0x8d,(byte)0xd5,(byte)0x4e,(byte)0xa9,
(byte)0x6c,(byte)0x56,(byte)0xf4,(byte)0xea,(byte)0x65,(byte)0x7a,(byte)0xae,(byte)0x08,
(byte)0xba,(byte)0x78,(byte)0x25,(byte)0x2e,(byte)0x1c,(byte)0xa6,(byte)0xb4,(byte)0xc6,
(byte)0xe8,(byte)0xdd,(byte)0x74,(byte)0x1f,(byte)0x4b,(byte)0xbd,(byte)0x8b,(byte)0x8a,
(byte)0x70,(byte)0x3e,(byte)0xb5,(byte)0x66,(byte)0x48,(byte)0x03,(byte)0xf6,(byte)0x0e,
(byte)0x61,(byte)0x35,(byte)0x57,(byte)0xb9,(byte)0x86,(byte)0xc1,(byte)0x1d,(byte)0x9e,
(byte)0xe1,(byte)0xf8,(byte)0x98,(byte)0x11,(byte)0x69,(byte)0xd9,(byte)0x8e,(byte)0x94,
(byte)0x9b,(byte)0x1e,(byte)0x87,(byte)0xe9,(byte)0xce,(byte)0x55,(byte)0x28,(byte)0xdf,
(byte)0x8c,(byte)0xa1,(byte)0x89,(byte)0x0d,(byte)0xbf,(byte)0xe6,(byte)0x42,(byte)0x68,
(byte)0x41,(byte)0x99,(byte)0x2d,(byte)0x0f,(byte)0xb0,(byte)0x54,(byte)0xbb,(byte)0x16
    };

    /**
```

```
     * Constructor for making a KeyManager with a non-specified key string.
     *
     * Uses a pre-determined key string of length 16. (16 chars = 16 * 8 bits per char =
128 bit size)
     * For Java and I guess in this project too, we are just doing 1 char = 8 bits
     */
    public KeyManager() {
        this.key = getByteKeyFromString("0123456789abcdef"); // THIS MUST BE A STRING OF
LENGTH 32
        this.roundKeys = generateRoundKeys(this.key);
    }

    /**
     * Constructor for making a KeyManager with a specified key string.
     *
     * @param keyString a string of length 16 to be used as the key (16 chars = 16 * 8
bytes per char = 128 bit size)
     */
    public KeyManager(String keyString){
        // Make sure that the size of the string is correct. If not, either cut it or add
characters.
        keyString = setCorrectStringKeySize(keyString);
        this.key = getByteKeyFromString(keyString);

        this.roundKeys = generateRoundKeys(this.key);
    }
```

Set-up environment

- CS-331-GROUP-5-PROJECT
    - Libraries
        - Jama-1.0.3.jar
    - Project
        - AES.java
        - AESTester.java
        - KeyManager.java
        - MainProject.java

- Challenges faced during implementation
    - One main challenge that we faced was simply just first understanding the functions used in AES to encrypt and decrypt
    - AES seems to be a little more complicated than other cryptographic methods (There are many different processes involved, including round keys, byte substitution, mixing columns, and shifting rows), which both allows it to be more secure, but also can make it a little more confusing to learn [3], which slowed down our code creation process a bit
    - Fortunately, as we continued through the project, and further implemented our program, we were able to gain a firmer understanding of the different functions

The project was developed as a group utilizing Github and Github Projects. Using Github Projects, we were able to outline specific parts of the project and group members

were able to identify which issues they were currently responsible for and mark each part as completed when done.

Results and Analysis

We evaluated our implementation by running a series of tests with different plaintext messages and keys. Test inputs included short messages such as `"hi"`, typical words like `"hello"`, sentences with spaces and punctuation, and longer strings that required multiple 16-byte blocks. For each test we entered a 16-character key, encrypted the message, and then immediately decrypted the resulting ciphertext. In all test cases the decrypted output matched the original plaintext, confirming that our encryption and decryption routines are functionally consistent and reversible for the inputs we support.

We also tested several edge cases. When the user enters a key shorter or longer than 16 characters, the `KeyManager` either truncates or pads the key string so that it always fits the required 128-bit size. Similarly, when the plaintext is not a multiple of 16 characters, our code pads the last block with null characters so that it can still be processed. These design choices make the program more robust and reduce the likelihood of runtime errors caused by invalid input sizes.

However, our results also highlight important limitations. First, the program uses a simple ECB-style block mode, which is known to be insecure for many real-world applications because identical plaintext blocks produce identical ciphertext blocks. Second, parts of our internal implementation are simplified for educational purposes and do not fully match the FIPS-197 AES standard, especially in how arithmetic is handled compared to true finite-field operations. Therefore, while the project successfully provides the structure and flow of AES encryption and decryption that may be used in a cybersecurity environment, it should not be used in the program's current state for handling sensitive data.

Discussion

Based on our results, this project met the objectives. Objective one was to create a demonstration of AES encryption and decryption. We believe that we successfully completed this objective as our program can be used by others to encrypt and decrypt a message of their choosing, and displays the steps that are taken to do so. With the time we had, we were able to create a program that displays the use of AES to users with minimal bugs.

Several insights were gained during this project. The first was how AES functions outside of theory. Due to the multiple round keys and the way each round is broken into smaller steps, it’s more complicated and takes more time to encrypt or decrypt than some of the other symmetric ciphers we learned about in class, such as Caesar or Vigenere. Another insight gained was the extent of transparency the program could give the user before it was no longer secure. AES follows the design principle of Open Design, so its

security does not come from the secrecy of its design.

However, it's important to note that a step by step demonstration of AES isn't a secure environment, since keys and encrypted/decrypted results are displayed automatically to the user. For learning purposes, it is beneficial for the user to know what the key and starting message are in plaintext, otherwise they'll be confused.

Besides the limitations mentioned in the results and analyses section, the program is limited in that all the output is expressed in plaintext, so the user does not see the bytes as represented in the S box or in any traditional example of AES.

Conclusion and Future Work

One major achievement that came from this project was understanding AES, which was also one of the lessons learned. AES is a complicated cipher, and learning about it caused the team to learn or relearn about several other concepts, such as basic linear algebra and using the byte datatype in code. It showed that not all symmetric ciphers are as simple as the Caesar Cipher. And since it's one of the most commonly used ciphers in the real world, something something something something something.

There are many ways that this project could be extended or improved in the future. One option would be to implement some kind user interface to make it easier for the user to interact with the program. For instance, the program is currently set up so that a user has to enter exactly 16 characters for the user generated key, and it won't move forward until that task is met. So as of right now, the user has to count out the 16 characters themselves before they submit, which can be frustrating, as the user has to enter a key each time they use the program. Creating a user interface that displays the number of characters entered as the user is typing would make the process more convenient and it would help the program meet the Principle of Least Astonishment. A user interface would also improve the readability of the program's output. Currently, the program outputs in one big column in the terminal. If output from the encryption rounds and decryption rounds could be side by side, it would be easier for the user to see the relationship between them.

Outside of the visual aspects of the program, it could be expanded to have more demonstration levels. While there are two options, they go from displaying information for every encrypting and decrypting step to displaying just the keys, final encryption, and final decryption. There are certainly levels in between those two options that would be useful for a user. For instance, the most helpful information for one debugging situation came from the final matrix from each round. Finally, the learning aspect of the program could be expanded as well. It could include other ciphers to walk through, from simple ones like Caesar to more complicated ones like RSA. Each cipher could have demonstrations of how it's used in the real world. For instance, AES could have a simulation of how a communication service would use it to encrypt and decrypt messages.

References

[1] Buchanan, William J. “Addition and Multiplication Tables in Galois Fields GF(2^m).” *Asecuritysite.com*, 2025. Notes and Youtube video explaining how a part of the MixColumns step works with multiplication of bytes. https://asecuritysite.com/principles/gf

[2] Chen, Li-An. “Advanced Encryption Standard (AES).” MATH 307, Boise State University, 2025. Lecture slides on how AES works which was used to help us understand more about AES. https://github.com/leohaenerpope/CS-331-Group-5-Project/blob/main/AESPDF.pdf

[3] National Institute of Standards and Technology. “Advanced Encryption Standard (AES).” *Federal Information Processing Standards Publication*, US Department of Commerce, 26 May 2001, updated 9 May 2023. https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197

```
public String decrypt(String encryptedData, String key) {
        // use key from key manager
        this.keyManager = new KeyManager(key);
        this.currentRoundNum = roundNum; // start at round 10 (final)

        String fullDecryptedData = "";
        for (int d = 0; d < encryptedData.length(); d += 24) {
            String dataBlock = encryptedData.substring(d,
Math.min(encryptedData.length(), d+24));
            while (dataBlock.length() < 16) {
                dataBlock += "\0"; // padding with null characters
            }

            byte[] cipherBytes = java.util.Base64.getDecoder().decode(dataBlock);

            // Load cipher into 4x4 matrix
            for (int i = 0; i < 16; i++) {
                int row = i % 4;
                int col = i / 4;
                dataArray[row][col] = (i < cipherBytes.length) ? cipherBytes[i] & 0xFF :
0;
            }
            dataMatrix = new Matrix(dataArray);
            AddRoundKey(); // final round key

            // inverse transformation, for final [inverse shiftrows + inverse subbytes]
            InvShiftRows();
            InvByteSubstitution();

            // run the decryption for round 1-9
            for (currentRoundNum = roundNum - 1; currentRoundNum > 0; currentRoundNum--)
{
                AddRoundKey();
                InvMixColumns();
                InvShiftRows();
                InvByteSubstitution();
            }
            AddRoundKey(); //round 0 key

            // convert matrix back into plaintext
            dataArray = dataMatrix.getArray();
            byte[] plainBytes = new byte[16];
            for (int i = 0; i < 16; i++) {
                int row = i % 4;
                int col = i / 4;
                plainBytes[i] = (byte) dataArray[row][col];
            }

            // convert plaintext into str and store in var
            String decryptedData = new String(plainBytes).trim();
            fullDecryptedData += decryptedData;
        }
        return fullDecryptedData;
    }
```